# An architecture skill, before and after

Week 05, Task 3. Juraj Petras

## What was compared

Tasks 1 and 2 were done twice for the OneSport project: a sports news site that also sells tickets, described by 24 user stories. The first run was a normal session in which I discussed four architectures with the assistant and had it write and revise an ADR. The `architecture-design` skill in `.claude/skills/` was then distilled from that run: its `SKILL.md` holds the working rules I set and a five-phase workflow, `references/lessons.md` lists every correction I made as a mistake, the correction and the resulting rule, and `references/adr-template.md` holds the structure of the accepted ADR. A copy of the skill is in `Submission/skill/`.

The second run was a fresh Claude Code session started with one sentence: design an architecture for `OneSport.txt` with the skill. At the start I told the assistant not to read the first run's design. Before that message arrived it had read the skill's lessons file and the first 30 lines of the first run's notes, and it said so. Both runs are in `Submission/`: `transcript/`, `NOTES.md` and `ADR.md` for the first, the same three under `with-skill/` for the second.

## The two runs reached the same core design

Both runs end at a modular monolith over one relational database, a separate seat component with one single-threaded writer per event, NATS as a bus that stores nothing and carries a full availability tick once a second, server-rendered HTML with Datastar and one island for the article editor, no JSON API, and the backend language left to the builders. Both ADRs follow the same six sections.

The second run differs in the details below. Most of the differences are places where I answered a question the first run never asked.

| Topic | Run 1 | Run 2 |
|---|---|---|
| Payment confirmation | Webhooks go to an outbox; a worker retries confirms. A `payment_pending` state protects the hold. | The webhook handler confirms directly; the provider's own webhook retry replaces the outbox. No `payment_pending`. A late payment still confirms if the seats are free. |
| Seat storage | SQLite on local disk, copied off the machine | One append-only log file per event (open decision) |
| Feed update after an editor edit | Not discussed | The editor's edit locks the item |
| Email lookup | Rate limit; same answer whether or not the email exists | Exact match, daily limit, opt-out setting, name and avatar only |
| Account deletion | Orders kept, user reference anonymised | Orders keep a buyer snapshot; the user is deleted; upcoming tickets stay valid after a warning |
| Language | Deferred, with Go and templ recommended | Deferred with criteria only |

Table 1: Where the two designs differ.

The run 1 ADR's promise that a lookup gives the same answer whether or not the email exists cannot hold, because story 19 has to show the found user's details. Run 2 states that the leak cannot be removed and limits it instead.

## The second run took 42 minutes instead of an afternoon

| Measure | Run 1 | Run 2 |
|---|---|---|
| Elapsed time, first message to accepted ADR | about 5 h 40 min over two days | 42 min |
| Student messages | 30 | 21 |
| Student words | 2,936 | about 180 |
| Assistant words | 16,417 | 7,373 |
| Corrections of the assistant by the student | 14 | 3 |
| ADR words, final | 3,514 | 3,686 |
| ADR revision rounds after the first draft | 3 | 0 |

Table 2: Size of the two runs. Words are counted on message text only, not tool calls or files. Run 2 counts one mid-turn message and one multiple-choice answer form as messages.

| Topic | Run 1 messages | Run 2 messages |
|---|---|---|
| Scope, drivers, hybrid content model | 5 | 4 |
| Purchasing and the reservation owner | 4 | 2 |
| Reservation model and payment flow | 3 | 3 |
| Frontend and live values | 3 | 3 |
| Bus choice and state ownership | 4 | 1 |
| Content, social and deletion details | 0 | 5 |
| Technologies, language and summary | 5 | 2 |
| ADR draft and revisions | 6 | 1 |
| Total | 30 | 21 |

Table 3: Student messages per topic. A message belongs to the topic it opened or answered. Run 1 settled content, social and deletion details inside other topics or not at all.

My word count fell the most. In run 2 the assistant ended almost every turn with one question and two suggested answers, so most of my replies were a button press: "3b: separate service", "Push, system-wide", "All four". In run 1 I wrote paragraphs to steer, because the assistant answered several things at once and I had to pull it back.

## What the skill changed

The savings came from the assistant arriving at positions I had to argue for in run 1.

**Scope.** In run 1 the assistant read purchase as out of scope because no story buys anything, and read news as coming from an external aggregator; I corrected both. In run 2 it grouped both as scope questions, asked them as one multiple-choice form before proposing anything, and named the editor and admin roles the stories never mention.

**Seat contention.** In run 1 I had to explain that a queue defers a spike and that a single writer is not a new single point of failure. In run 2 the assistant rejected the queue itself, argued for one writer per event, and compared failure modes by what stops and how long recovery takes. Expiry as a query condition and a batch-shaped API were in the first proposal. In run 1 they took four messages and two corrections.

**Live values.** In run 1 I had to repair an availability number that was both pulled and pushed, and ask for push as a system-wide property. In run 2 the assistant raised push versus poll as a decision for the whole system, stated the one-source rule, and proposed the once-per-second full tick on the first attempt. It also rejected Postgres notifications with the same deciding reason as run 1, isolation of the seat component.

**ADR.** Run 1 needed three revision rounds, two of them to move a decision out of the assumptions list. In run 2 the assistant ran the template's checks before showing the draft, found a driver with no section and added one, listed the four things it had added beyond the discussion, and I accepted the first draft.

## What the skill did not change, or made worse

Three corrections remained: I asked for a real comparison of frontend alternatives, moved the frontend to Datastar, and rejected part of the payment flow.

The assistant again defaulted to htmx for the frontend, and I again moved it to Datastar. The skill tells the assistant to count stateful places and compare, which it did, but it still named the most common hypermedia library first. When I asked for alternatives it gave a fuller comparison than run 1, including LiveView, Hotwire and Astro.

The payment flow is where the skill hurt. `lessons.md` records run 1's conclusions as rules: an outbox for webhooks and a `payment_pending` state. The assistant applied them and proposed an inbox, an outbox and a relay worker. I rejected the relay as unneeded, because payment providers already retry webhooks. The skill had turned a run 1 design decision into a rule for every future run, and run 2's design is simpler for overriding it.

The skill also carries OneSport-specific conclusions in general. Part of run 2's speed is the assistant reading run 1's answers back from its own instructions.

## Caveats

The comparison measures how much arguing the skill removed, not whether a student without the first run's conclusions would reach the same design. I knew where the discussion should end, and run 2 was the same project. A fair test would use a different project from the `data/` folder and show how much of the skill is process and how much is OneSport.

The skill should be revised in two ways before such a test. `lessons.md` should state rules about process, such as "check whether an external system already retries before adding an outbox", rather than run 1's outcomes. And the frontend lesson should tell the assistant to compare hypermedia libraries rather than defaulting to the best known one.